rbus

локальная высокопроизводительная шина сообщений для автомобильных и робототехнических систем

Проблема

Нужно передавать данные между процессами в рамках одного устройства

- Универсальные робототехнические фреймворки (по типу ROS 2) часто рассчитаны на широкий класс систем
- В них есть сетевые сценарии, разные режимы доставки (QoS-ы) и большое число runtime-параметров
- Есть сценарии, где такая универсальность не требуется - например, когда процессы живут физически в одном месте

Отсюда возникла идея сделать локальный транспорт, который обеспечивает низкую задержку и низкую нагрузку на CPU

Цель проекта

Сделать такой транспорт

Поскольку основным паттерном взаимодействия в такого рода системах является publish/subscribe, то основное внимание в работе будет уделено именно ему

Что требуется:

- **Производительность** — активное использование shared memory, loaned payload без копирования, блокирующий и неблокирующий пути передачи с минимальным количеством syscall-ов
- **Простая модель и локальный scope** — простой и понятный API, одно устройство, один daemon, без попытки сделать универсальный middleware
- **Надёжность** — освобождение ресурсов при падении или штатной остановке процессов, restart daemon-а не должен ломать уже настроенный путь для данных
- **Простая и информативная диагностика** — cli-утилиты, stack trace-ы и т. д.

Границы проекта

Что не входит в текущий этап

- сетевой транспорт
- сложные политики доставки
- async/callback API
- serde-схемы
- C/Python bindings
- автоматический запуск daemon-а

Архитектура

Daemon строит связи, payload идёт напрямую

Control plane

Publisher ↔ **rbusd** ↔ **Subscriber**

- REGISTER / ATTACH / INSPECT идут через rbusd по Unix SOCK_SEQPACKET
- FD передаются через SCM_RIGHTS
- daemon владеет registry и lifecycle

Data plane

Publisher → **memfd MAP_SHARED** → **Subscriber**

- daemon не копирует и не маршрутизирует payload
- после attach hot path работает через shared memory

Flow

Publisher path

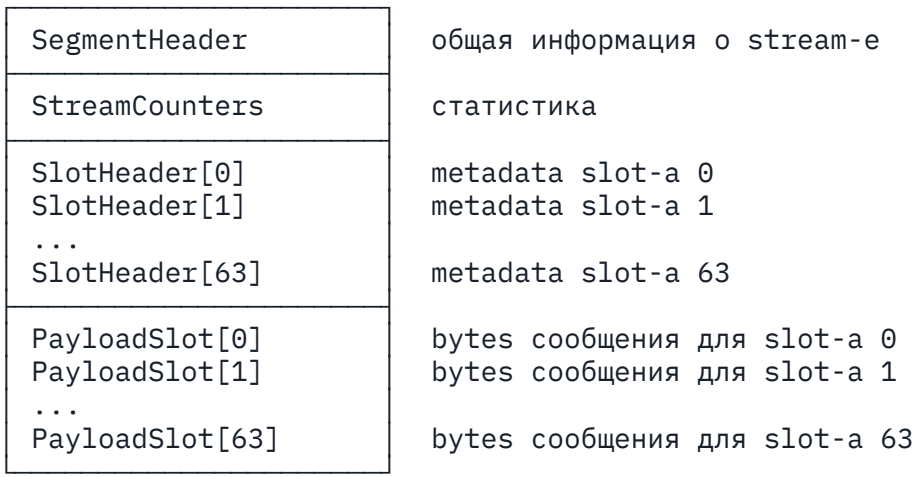
Bus::connect()
↓
bus.publisher(topic, TopicConfig::new(max_size, depth))
↓
rbus отправляет REGISTER_PUBLISHER в rbusd
↓
rbusd создаёт memfd stream segment
↓
rbusd делает ftruncate до нужного размера
↓
rbusd инициализирует header/ layout
↓
rbusd передаёт FD publisher-у через SCM_RIGHTS
↓
publisher mmap-ит segment

Subscriber path

bus.subscriber(topic)
↓
rbus отправляет REGISTER_SUBSCRIBER
↓
rbusd выдаёт subscriber_id и topic wait FD
↓
subscriber вызывает ATTACH_STREAMS
↓
rbusd передаёт FD всех stream-ов этого topic-а
↓
subscriber mmap-ит эти stream segment-ы

Stream segment

Stream — это per-publisher message stream, реализованный как shared-memory ring segment



- Stream — это SPMC ring, каждый publisher получает свой segment и пишет только в него
- Subscriber читает stream-ы round-robin, поэтому порядок между publisher-ами — это порядок наблюдения subscriber-ом

```
topic "/camera/front/image"
├── stream #1 от publisher #1
├── stream #2 от publisher #2
└── stream #3 от publisher #3
```

Гарантии hot path

Главные обещания data path

0

heap allocations: loan + publish

0

syscalls/message: non-blocking publish/try_recv

0

payload copies в loaned path

Как проверяется

- custom allocator tests
- marker-based strace smoke
- loom-модели atomic protocol
- layout tests на cache-line counters

Технологии

Минимальный стек

- **Rust 2024** — warnings=deny, dead_code=deny
- **Linux IPC** — memfd_create, ftruncate, mmap MAP_SHARED, SCM_RIGHTS, Unix SOCK_SEQPACKET
- **Синхронизация** — futex для блокирующего пути, acquire/release atomics для slot protocol
- **Бенчмарки** — perf stat, strace

Бенчмарки

Окружение и режимы нагрузки

- Ubuntu 26.04 LTS, Linux 7.0
- Intel Core i7-11700F

Режимы

- saturated / busy-poll: 4 publisher-а, 8 subscriber-ов
- payload: 64 B, 4 KiB, 64 KiB, 1 MiB
- paced / 30 Hz: controlled-load latency
- low-CPU recv: 5 секунд, 150 сообщений, 1P1S

Результаты: throughput

Saturated busy-poll, 4P8S, payload 64 B



5.7x

loaned-fill vs unix socket

0

syscalls/message в rbus в режиме busy-poll

0

allocations/message в rbus в режиме busy-poll

Результаты: большие payload

Saturated busy-poll, logical/effective delivery GiB/s

effective delivery = payload_size × delivered_messages/s, с учётом fanout; это не physical DRAM bandwidth

Payload	loaned	publish	unix	x
4 KiB	53.8	59.1	8.2	6.5x
64 KiB	120.2	92.1	21.8	5.5x
1 MiB	146.1	154.1	15.6	9.4x

x = rbus loaned-fill / unix-socket

Результаты: задержка под нагрузкой

Paced 30 Hz, 1P1S, busy-poll, p99 transport latency

Payload	loaned	publish	unix	x
64 B	1.2 us	2.4 us	126.7 us	105x
4 KiB	8.0 us	9.2 us	151.7 us	19x
64 KiB	23.5 us	23.8 us	276.1 us	12x
1 MiB	23.2 us	110.2 us	1615.5 us	70x

x = unix-socket p99 / rbus loaned-fill p99

- Этот режим показывает нижнюю границу latency
- Он использует busy-poll и почти целое ядро на worker path
- Metadata-only не используется как основной latency-result

Результаты: низкий CPU в recv

Paced 30 Hz, 1P1S, wait-mode recv

64 B

- rbus loaned-fill: p99 81 us, 0.004 cores
- unix-socket: p99 103 us, 0.001 cores
- вывод: rbus лучше по latency, Unix дешевле по CPU

1 MiB

- rbus loaned-fill: p99 231 us, 0.014 cores
- unix-socket: p99 1369 us, 0.019 cores
- вывод: rbus лучше по latency при сопоставимом CPU

Результаты: корректность

Не скрываем ошибки доставки

0

warning_count в выбранных строках

0

skipped_messages

0

publish deadline misses

Тесты

- cross-process data plane tests
- examples suite
- subscriber crash с удержанным MessageRef
- publisher crash
- rbusd restart

Что получилось хорошо, а что - нет

Хорошо

- payload не проходит через daemon
- zero-copy при loaned path
- высокий throughput на локальной машине
- микросекундные задержки в busy-poll режиме
- низкий расход CPU в блокирующем режиме (через futex)

Не очень хорошо

- жёсткая привязка к Linux-specific IPC примитивам
- API пока только для Rust-а
- нет не SHM-транспорта
- один stream ограничен 63 subscriber reader bits (?)
- низкий расход CPU (в блокирующем режиме) на малых payload не всегда дешевле Unix socket

Выводы

Цель достигнута

Реализован локальный message bus, который было бы интересно довести до ума и внедрить в кейсах где важна скорость и дешевизна передачи данных

Следующие шаги

- посмотреть что будет при реальных нагрузках в real-time системах
- сравнить с другими транспортами production уровня - по типу iceoryx2 [https://iceoryx.io/]
- упростить и стабилизировать API
- сделать C++ binding-и и адаптеры к ROS2-системам
- ...